

CPS506 Lab 8

Rust: Structs and Traits

Preamble

In this lab you will implement a couple traits for a custom type. This will be reminiscent of Lab 6, where you did something similar with Haskell type classes.

Lab Description

In the `main.rs` source file, you will implement a custom data type called **Poly**. This type is nothing more than a struct containing a vector of integers:

```
struct Poly {  
    coeffs: Vec<i32>,  
}
```

This vector represents a series of polynomial coefficients, where the index of the vector is the degree of that term. For example, the vector `<1, 2, 3>` represents the polynomial $1x^0 + 2x^1 + 3x^2$. You will implement several operations over **Poly**, each of which is described below.

The == operator:

Two Polynomial are considered equal if their vectors are equal (elements are the same).

The + operator:

The sum of two Polys is defined component-wise. That means if *a*, *b* are two polynomials then we define *c* = *a* + *b* as *c*.coeffs[i] = *a*.coeffs [i] + *b*.coeffs[i].

This method should be able to handle Polys of different lengths. When adding two Polys of length *m* and *n*, where *m* > *n*, the resulting list should be of length *m* and the higher elements of the shorter list is assumed to be zero. For example, for the following *a*, *b*:

```
a = <3,2,2,1,4>  
b = <2,4,1>  
a+b = <5,6,3,1,4>
```

The – operator:

Like the + operator, but this time performing element-wise subtraction. For example, for the above *a*, *b* we have:

```
a-b = <1,-2,1,1,4>
```

The * operator:

Multiplication between two Polys is performed the same way polynomial multiplication would be performed mathematically. Realize that the length of the resulting Poly will be longer, because multiplying the highest degree terms together results in a degree that is the sum of the two. E.g. $a^2 * b^3 = (ab)^5$.

Thus, the earlier example evaluates as follows. Work it out on paper to convince yourself.

```
a = <3,2,2,1,4>
b = <2,4,1>
a*b = <6,16,15,12,14,17,4>
```

Multiplication can also be performed between a polynomial and an integer. In this case, simply multiply that integer by each element in the polynomial.

Implementation Details

To help you get started, template code snippets are given below for each operation (these are in the cargo source file as well). In your main.rs source file, your job is to fill in these **impl** blocks. You must also use the following namespaces:

```
use std::fmt;
use std::ops;
use std::cmp;

impl ops::Add<Poly> for Poly {
    type Output = Poly;
    fn add(self, other: Poly) -> Poly {
        // Your + code here
    }
}

impl ops::Sub<Poly> for Poly {
    type Output = Poly;
    fn sub(self, other: Poly) -> Poly {
        // Your - code here
    }
}

impl cmp::PartialEq for Poly {
    fn eq(&self, other: &Self) -> bool {
        // Your == code here
    }
}
```

```

impl ops::Mul<Poly> for Poly {
    type Output = Poly;
    fn mul(self, other: Poly) -> Poly {
        // Your * code here
    }
}

impl ops::Mul<i32> for Poly {
    type Output = Poly;
    fn mul(self, other: i32) -> Poly {
        // Your * code here
    }
}

```

Submission

Labs are to be completed and submitted *individually*. Submit your **main.rs** source file containing all completed functions on D2L, under the submission for Lab 8.